

Unidade 2: JavaScript

A linguagem de **marcação HTML** deve ser usada especificamente para estruturar documentos. Já questões de **estilo** deve ficar a cargo da **folha de estilo (CSS)** como já exposto neste material.

Outro ponto é a inserção de interatividade em uma página HTML. Para isso, usa-se linguagens de programação.

Dentre as linguagens existentes, uma linguagem que pode ser usada para adicionar interatividade é a linguagem de script que roda no lado do cliente (**client side**) que visualiza a página, denominada JavaScript.

Criada por **Brendan Eich** e lançada juntamente com o navegador **Netscape Navigator 2.0** em 1996, permite o desenvolvedor adicionar interatividade a uma página Web.

A linguagem JavaScript depende exclusivamente do navegador do usuário que está visualizando a página. Sendo assim, as funcionalidades e os recursos são suportados pelas máquinas de interpretar scripts denominados **engines**. Por exemplo, o navegador Firefox tem o engine **SpiderMonkey** escrito na linguagem C/C++, o **Google Chrome** tem o engine **V8** escrito na linguagem C/C++ e existem vários outros engines.

Assim a interatividade é provida pelo navegador não sendo necessário depender de requisições ao servidor.

2.4.7 Funcionalidades

Ao usar a linguagem JavaScript o desenvolvedor pode **manipular e alterar o conteúdo** da página Web, **alterar características do navegador** e interagir com os formulários

criados em HTML, podendo acessar os campos e adicionar validações por exemplo.

2.4.8 Inserção de JavaScript no HTML

A linguagem JavaScript é incluída dentro de uma página Web. Para isso, deve ser usado um elemento denominado **script**. Tal elemento tem a função de incorporar um script ou invocar um arquivo de extensão **“.js”** para dentro do corpo da página Web.

O elemento **script** tem um atributo denominado **src**, este **atributo** tem a finalidade de **indicar o caminho que o script está localizado no servidor por meio de uma URL**.

No **HTML5**, o elemento `script` é codificado da seguinte forma:

```
<script src="arquivo.js"></script>
```

A forma de codificação apresentada permite chamar arquivos de **extensão “.js”** para dentro da página Web. Além dessa forma é possível incorporar o **script dentro do HTML**. Ou seja, entre as tags `<script>...</script>` é possível definir o código a ser implementada na página, como apresentado a seguir:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8"/>
    <title>Incorporado</title>

    <script>
      alert('Olá Mundo!');
    </script>
  </head>
  <body>
  </body>
</html>
```

Uma outra forma também usada é a inserção do JavaScript de **forma inline**. Trata-se de uma forma de inserir o script diretamente na seção **body**, misturando com os elementos de estrutura do HTML, como apresentado a seguir:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8"/>
    <title>Inline</title>
  </head>
  <body>
    <button type="button" onclick="alert('Olá
Mundo!');">Olá</button>
  </body>
</html>
```

2.4.9 Código JavaScript

Adicionar um script de forma **inline**, **externa** ou **incorporado** é um primeiro passo para criação de scripts. Entretanto, como deve ser escrito o código usando a linguagem JavaScript ?

O primeiro ponto que deve ser considerado é que a linguagem JavaScript é sensível em relação a caixa de texto – *case sensitive*.

Em outras palavras, **nomes de variáveis** com letras **minúsculas** e **maiúsculas** são diferentes, assim como escrever as palavras reservadas para criar o código também faz diferença.

Exemplo:

Total é diferente de **total**
document.Write() (gera erro) diferente de **document.write()**

Faz parte da sintaxe da linguagem a possibilidade de criar comentários no código, garantindo esclarecimento de pontos críticos da programação ao desenvolvedor.

Os comentários podem ser colocados em uma única linha e múltiplas linhas, veja o código a seguir:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Comentários</title>
    <meta charset="utf-8"/>
    <script>
      // Comentário de uma linha
      <!-- Comentário em linha única
      /* Comentário para múltiplas linhas*/
      alert("Inserir comentário!");
    </script>
  </head>
  <body>
  </body>
</html>
```

Para inserir algum tipo de regra ao script é necessário recorrer as declarações e estruturas de controle, pois script nada mais são do que sequencias de instruções com base em uma sintaxe rígida.

O código em JavaScript pode ser escrito tudo em uma mesma linha, ou em linhas separadas. Sendo que quando o código é escrito em uma mesma linha é necessário usar o **ponto-e-vírgula** para separar o código.

No entanto, o uso de ponto-e-vírgula é facultativo quando o código está escrito em linhas separadas.

Uma outra característica é o espaço em branco que é ignorado quando inserido entre a declaração de variáveis, funções e números, observe o código a seguir:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Declaração</title>
    <script type="text/javascript">
      graduação = "BSI"; curso= "DSW2014";
      alert("Olá pessoal!");
    </script>
  </head>
  <body>...</body>
</html>
```

A criação de variáveis é importante em qualquer linguagem, principalmente porque é através das variáveis que os dados são manipulados e as regras podem ser incluídas nos scripts.

As variáveis e as constantes em JavaScript não possuem definição de tipos, pois o tipo fica implícito no valor que a variável ou a constante recebe.

Para criar uma variável nesta linguagem deve ser considerado algumas características relacionadas a criação do nome, ou seja, o nome da variável deve começar com uma letra ou caractere underline (_) ou cifrão (\$). Na sequência pode seguir letra maiúscula ou minúscula, números, etc.

A **linguagem JavaScript** apresenta os seguintes tipos de dados: *inteiros*, *decimais*, *booleanos*, *strings*, *arrays* e *objeto*.

Um **tipo de dado inteiro** pode ser entendido com número fracionários ou inteiro, veja as possíveis variáveis recebendo número fracionários: `a=4.5; b=90; .`

O **tipo de dado booleano (boolean)** recebem o valor *true* ou *false* que devem ser escritos em letras minúsculas. Por exemplo: `a=true; b=false; .`

Agora o **tipo de dado string** trata-se de um ou mais caracteres envolvidos por aspas duplas ou simples. Por exemplo: `graduacao="BSI"; curso="DSW"; .`

O **tipo de dado string** pode ser encarado como um objeto, podendo ser aplicado alguns métodos para manipular o texto armazenado. Alguns métodos contêm quantidade de caracteres `length`, colocar todas os caracteres em maiúsculo `toUpperCase()`, colocar todas os caracteres em minúsculo `toLowerCase()`, entre outros.

O **tipo de dado array** pode guardar qualquer tipo de dado, incluindo expressões, objetos e outros tipos de array. O array inicia-se com um índice sequencial começando do zero e tem a seguinte sintaxe:

```
carros = ["monza", "celta", "sonic"];
```

Para guardar mais de um tipo de dado, segue a seguinte sintaxe:

```
arrayMisturado = ["laranja", 2002, "casa", a+b, [1,2,3,4,5]];
```

A variável dependendo do escopo no código JavaScript pode ter o papel de **constante** ou **variável**. Geralmente para declarar uma variável pertencente ao **escopo local** usa-se o prefixo **var** antes da variável, veja o exemplo a seguir:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Escopo</title>
    <meta charset="utf-8"/>
    <script>
      numero = 100;

      funcaoUm = function() {
        var numero = -20;
        alert(numero);
      }

      //Imprimr o valor -20
      funcaoUm();

      funcaoDois = function() {
        alert(numero);
      }

      //Imprimir o valor 100
      funcaoDois();

      //Imprimir o valor 100
      alert(numero);
    </script>
  </head>
```

```
<body>
</body>
</html>
```

No código acima, o **valor da constante número 100** é apresentado dentro de uma mensagem quando a **função Dois** ou o **alert(numero)** são invocados. Isso acontece porque não ocorreu nenhuma alteração de valor. No entanto, ao invocar a chamada da **função Um** é apresentado o valor -20 porque dentro da função é criado uma variável local com o nome **numero**.

Para amarrar as variáveis e criar as regras precisa ser estudado como são codificados a estruturas condicionais e de loop na linguagem JavaScript.

As condições são criadas com uso do **if** que deve conter uma expressão seguido por chaves de abertura e fechamento, veja o exemplo a seguir:

```
var media = 10;
if (media <= 10) {
    alert("Aluno reprovado!");
} else {
    alert("Aluno aprovado!");
}
```

Vale lembrar que `(media <= 10)` é uma expressão, e por se tratar de uma expressão pode ser combinado os diversos operadores que existem na linguagem que são apresentados no próximo tópico.

Em resumo , a estrutura condicional pode ser codificada da seguinte forma:

```
if ([expressao]) {
    /* Algo deve acontecer se a
       expressão resultar
       em uma verdade */
} else if ([expressao]) {
    /* Algo deve acontecer se a
       expressão resultar
       em algo falso. */
} ....
```

Além do `if`, existe a estrutura `switch` na linguagem JavaScript. A estrutura `switch` funciona como um chaveamento em que uma determinada variável é usada para identificar quais das condições do `switch` deve ter o código executado, veja a estrutura da sintaxe a seguir:

```
switch ([expressão]) {  
    case resultado :  
        //algo aqui;  
        break;  
    case resultado :  
        //algo aqui;  
        break;  
    ...  
    default:  
        //comportamento padrão  
}
```

As estruturas para controlar o loop estão mais relacionadas com o **for**, **while** e **do/while**. Em todas as estruturas é possível presenciar um condição para que o loop seja executado. Vale lembrar que o **do/while** tem uma característica diferente das demais, ou seja, a interação é executado pelo menos uma única vez.

A seguir são apresentados a sintaxe dos três tipos de estrutura de loops:

for

```
<!DOCTYPE html>  
<html>  
    <head>  
        <title>FOR</title>  
        <script type="text/javascript">  
            var contar = "";  
            for(var i=0; i<10; i++) {  
                if ( i < 9) {  
                    contar += i+", ";  
                } else {  
                    contar += i;  
                }  
            }  
            document.write(contar);  
        </script>  
    </head>  
    <body>  
    </body>
```



```
</html>
```

while

```
<!DOCTYPE html>
<html>
  <head>
    <title>WHILE</title>
    <script type="text/javascript">
      var contar = "";
      var i = 0;
      while(i < 10) {
        if (i < 9) {
          contar += i + ", ";
        } else {
          contar += i;
        }
        i++;
      }
      document.write(contar);
    </script>
  </head>
  <body>
  </body>
</html>
```

do/while

```
<!DOCTYPE html>
<html>
  <head>
    <title>DO/WHILE</title>
    <script type="text/javascript">
      var contar = "";
      var i = 0;
      do {
        if ( i < 9) {
          contar += i+", ";
        } else {
          contar += i;
        }
        i++;
      } while(i < 10);
      document.write(contar);
    </script>
  </head>
  <body>
  </body>
</html>
```

Após apresentado a sintaxe, a forma como criar comentários, a forma como o código deve ser escrito, a criação

de variáveis, os tipos de dados, o comportamento do escopo da variável em relação a declaração no código, a criação de estruturas condicionais e de loops é necessário conhecer os operadores que vão auxiliar na construção da lógica de programação do script.

2.4.10 Operadores

Ao estudar uma linguagem de programação é fato que operadores mais comuns tais como +, -, *, /, <, >, <=, >= estão presentes em grande parte das linguagens.

Em JavaScript é válido destacar que além da função de incremento que o operador + tem, este mesmo serve para concatenar strings. Exemplo: `a = "4" + "5";`.

Além dos operadores mais comuns, citado anteriormente, o JavaScript apresenta os seguintes operadores:

Operador	Finalidade
==	Igualdade: compara se os símbolos são iguais – retorna true se os dados são iguais e false , caso contrário. Ao comparação é feita sem levar em conta o tipo de dado.
!=	Não igualdade: executa de forma contrário a função de igualdade.
&&	E (lógico): operador direito e esquerdo devem ser iguais para retornar true , caso contrário retorna false .
	OU (lógico): retorna true se um dos operandos for verdadeiro.
[]	Indexação de arrays.
()	Função.
?:	Operador ternário.
=	Atribuição de valor.
+=, -=, *=, /=	Atribuir valor executando alguma operação.
++, --	Pré ou Pós / Incremento (++) ou Pré ou Pós decremento (--) .
===	Identidade: compara os símbolos considerando

!===

!

o tipo de dado. Retorna **true** se forem iguais, e **false** caso contrário.

Não identidade: executa de forma contrário a função de identidade.

Not: colocado antes de um operador nega a resposta.

Os operadores auxiliam o desenvolvedor na construção das regras que são aplicadas na linguagem JavaScript. Assim, o código JavaScript é composto por variáveis regidas por operadores que são controladas por estruturas condicionais ou loops.

O código por sua vez pode ser organizado dentro de funções e essas funções são criadas dentro dos arquivos de extensão “.js”, ou então, incorporado dentro da página Web.

2.4.11 Função

No desenvolvimento de código é normal existir trechos que se repetem. Os trechos por sua vez podem fazer parte de uma função.

A função trata-se de um bloco de código (corpo da função) que pode realizar cálculos complexos à manipulação de dados do script.

O uso da função ajuda a organizar o código de um script garantindo a manutenibilidade e legibilidade do código. Geralmente a função é criada com um nome que indica a ação a ser executada pela função.

Na linguagem JavaScript uma função pode ser criada de diferentes maneiras. A primeira dessas maneiras é a **função estática** que para ser executada precisa ser realizado a chamada direta da função, veja a seguir como codificar uma **função estática**:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8"/>
```

```

        <title>Função estática</title>
        <script>
            function ola() {
                alert('Bem-vindo ao curso DSW
do BSI!');
            };
        </script>
    </head>
    <body>
        <button type="button" onclick="ola()">Função
estática</button>
    </body>
</html>

```

Outra forma de criar função é denominada de **função anônima ou dinâmica**, em que com o uso da declaração *new* seguida do construtor.

```

<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8"/>
        <title>Função Anônimo</title>
        <script>
            var calculaAreaTriangulo = new
Function("b", "h", "return b*h/2;");
        </script>
    </head>
    <body>
        <button
                                type="button"
onclick="alert(calculaAreaTriangulo(5,10));">Executar
função</button>
    </body>
</html>

```

Além da **função anônima ou dinâmica** e a **função estática**, existe a função denominada **sintaxe literal**. A **função sintaxe literal** é conhecida como **funções expressões** e pode ser codificada da seguinte forma:

```

<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8"/>
        <title>Função Sintaxe Literal</title>
        <script>
            var calculaAreaTriangulo =
function(b,h) {
                return b*h/2;
            };
        </script>
    </head>

```

```
<body>
    <button                                type="button"
onclick="alert(calculaAreaTriangulo(5,10));">Executar
função</button>
</body>
</html>
```

As funções podem além de realizar cálculo e manipular dados, retornar valores. O retorno é feito com o uso da palavra **return** no corpo da função como nos exemplos das funções anônimas e de funções expressões. Uma função pode retornar um array, objeto e outros tipos de valores tais como texto ou número.

2.4.12 Caixas de diálogos

A linguagem JavaScript permite criar **caixas de diálogos** do tipo **pop-up** permitindo coletar ou apresentar informações ao usuário.

Para apresentar uma caixa de diálogo com informação basta seguir a seguinte sintaxe:

```
<!DOCTYPE html>
<html>
    <head>
        <title>Alerta</title>
        <script>
            alert("Teste");
        </script>
    </head>
    <body>
    </body>
</html>
```

Além de apresentar informações para o usuário é possível permitir alguma interatividade com as caixas de diálogo no JavaScript. Para isso, com o comando `confirm` o usuário pode selecionar se deseja aceitar ou cancelar a operação, veja o exemplo a seguir:

```
<!DOCTYPE html>
<html>
    <head>
```

```

<title>window.confirm</title>
<meta charset="utf-8"/>
<script>
    function carregar() {
        valor = confirm("Por favor,
clique em qualquer botão!");
        if (valor) {
            alert("Clicou no botão
OK!");
        } else {
            alert("Clicou no botão
CANCEL!");
        }
    }
</script>
</head>
<body onload="carregar();">
</body>
</html>

```

No código apresentado é criado uma **função carregar()** que é executada quando a página é carregada. Ou seja, o evento **onload** carrega a função e executa uma caixa de diálogo para o usuário clicar em uma das opções, **OK** ou **CANCEL**.

Outra forma de interagir com o usuário é solicitar que seja informado algum dado e isso pode ser feito com o uso do comando prompt da linguagem JavaScript. Veja o exemplo a seguir:

```

<!DOCTYPE html>
<html>
    <head>
        <title>window.prompt</title>
        <meta charset="utf-8"/>
        <script>
            valor = prompt("Qual o seu
nome!", "nome");
            alert(valor);
        </script>
    </head>
    <body>
    </body>
</html>

```

As caixas de diálogo permitem interação do usuário com a página web apresentando informação ou solicitando algum dado a ser processado pelo site.

2.4.13 Eventos

As caixas de diálogo apresentam alguma interatividade com o usuário que navega na página Web.

No entanto, um outro recurso muito importante para a linguagem JavaScript são os eventos que permitem a interação com os elementos de um documento HTML.

Por exemplo, ao clicar em um botão da página é possível acionar uma função criada na linguagem JavaScript e efetuar algum tipo de cálculo, sendo o cálculo apresentado na tela.

Ou então, com base em uma série de dados informados em caixas de texto (inputs html) realizar algum tipo de cálculo após clicar em um botão na página.

Os eventos permitem a interatividade do código JavaScript com os documentos HTML e existem vários tipos de eventos que podem ser associados aos elementos, por exemplo: *onload*, *onblur*, *onchange*, *onfocus*, *onkeydown*, *onkeyup*, *onmouseover*, *onmouseout*, *onclick*, *ondblclick*, *onunload*, *onresize*, entre outros.

2.4.14 Prática

Na prática do tópico da linguagem JavaScript vai ser criada a página do “**Fale Conosco**” do exemplo do Sistema de Gerenciamento de Notícias. Sendo assim, o desenvolvedor vai ser guiado a construir um formulário com os campos nome, assunto e mensagem. A ideia é quando o usuário clicar no botão para enviar uma mensagem, dispare um alerta caso os campos estejam com o valor em branco.

A ideia dessa prática é mostra como a linguagem JavaScript pode ajudar o desenvolvedor a criar validações ou outro tipo de interação.

Entretanto, vale apresentar nesta prática um comando ainda não comentado. A linguagem JavaScript permite que o desenvolvedor crie algumas validações no documento HTML. Para ter acesso ao campos de um formulário por exemplo, o desenvolvedor pode recorrer ao **DOM (Document Object Model)** que permite acessar todos os elementos de uma página Web.

Na linguagem JavaScript pode ser usado o comando **document.getElementById** para recuperar o valor de um campo de um formulário e atribuir o valor deste campo a uma variável local e na sequência realizar as validações necessárias.

O comando **document.getElementById(nome do valor atribuído ao id do elemento)** recupera o objeto input (tag HTML mencionada anteriormente) de um determinado ID. Assim, para recuperar o valor deste campo input precisa complementar o comando o **document.getElementById(nomeID).value**.

O formulário criado fica com o *layout* igual ao da Figura 15, considerando a visualização no Firefox.

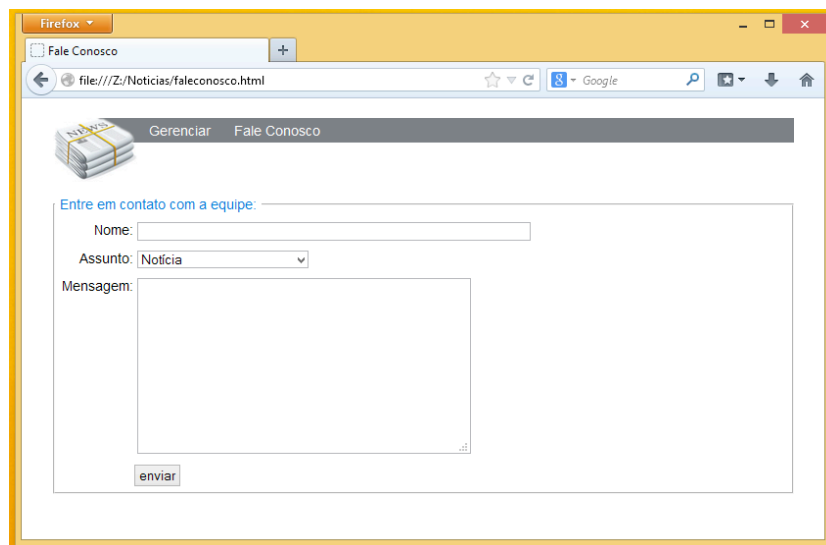


Figura 15 - Formulário "Fale Conosco"

Para gerar a página apresentada na Figura 15 foi criado o código **HTML, CSS e JavaScript** apresentado a seguir.

arquivo: faleconosco.html

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8"/>
        <title>Fale Conosco</title>
        <!--Garante o suporte correto do HTML5 para
o Internet Explorer-->
        <!--[if lt IE 9]>
        <script
src="http://html5shiv.googlecode.com/svn/trunk/html5.js">
</script>
        <![endif]>

        <link                rel="stylesheet"
href="_css/estilo.css" type="text/css"/>
        <!-- Código JavaScript para obrigar que o
usuário só informe algum valor nos campos -->
        <script>
            /*
                A função validar tem como corpo
do código duas variáveis locais com o nome de nome e
mensagem.

                Cada variável recebe o valor do
formulário, para isso é usado o comando
document.getElementById(nome do ID do campo) seguido do
ponto value.

                Para realizar a comparação é
usado a estrutura condicional IF e o operador de
comparação igual == .
            */
            function validar() {
                var        nome                =
document.getElementById("nome").value;
                var        mensagem            =
document.getElementById("mensagem").value;
                if (nome == "") {
                    alert("Deve ser
informado um valor para o nome!");
                } else if (mensagem == "") {
                    alert("Deve ser
informado um valor para a mensagem!");
                }
            }
        </script>
    </head>
    <body>
        <header>
            <span                id="logotipo"></span>
```

```

        <nav>
            <ul id="menu">
                <li><a
href="#">Gerenciar</a></li>
                <li><a
href="faleconosco.html">Fale Conosco</a></li>
            </ul>
        </nav>
    </header>
    <section>
        <article>
            <form action="#" method="POST">
                <fieldset>
                    <legend>Entre em
contato com a equipe: </legend>
                    <p>
                        <label
for="nome">Nome:</label>
                        <input
type="text" id="nome" name="nome" class="texto" />
                    </p>
                    <p>
                        <label
for="assunto">Assunto:</label>
                        <select
name="assunto">
                            <option selected>Notícia</option>
                            <option>Solicitar retirada de noticia</option>
                        </select>
                    </p>
                    <p>
                        <label
for="mensagem">Mensagem:</label>
                        <textarea
class="textarea" name="mensagem" rows="10" cols="40"
id="mensagem"></textarea>
                    </p>
                    <p><label></label><input
value="enviar" onclick="validar();" /></p>
                </fieldset>
            </form>
        </article>
    </section>
</body>
</html>

```

arquivo: estilo.css

```

/*
Define a margem e o espaçamento com o valor igual a zero.

A propriedade margin refere-se a margem e ao aplicar o
valor igual a zero tanto no topo, embaixo, lado direito e
esquerdo da imagem a margem é igual a zero.

```

```

A propriedade padding define um espaçamento do conteúdo
com as bordas ao retorno do elemento. Neste caso, quando
definido 0 significa que todos os elementos tem o padding
tanto em cima, abaixo, lado direito e lado esquerdo igual
a zero.
*/
* {
    margin:0;
    padding:0;
}

/*
Define a fonte da página em 88% e fonte igual a Arial,
Helvetica e Sans-serif
*/
body {
    font:88% Arial, Helvetica, sans-serif;
}

/*
A propriedade text-decoration pertencente a tag "a" que
define um link no documento HTML, quando atribuída com o
valor none retira o sublinhado.
*/
a {
    text-decoration: none;
}

/*
A propriedade list-style-type retira os bullets de uma
lista. Entende-se por bullets o desenho ou símbolo que
fica ao lado esquerdo de uma lista.
*/
ul,ol {
    list-style-type:none;
}

/*
A propriedade width define a largura do cabeçalho
representado pela tag "header" do HTML5 em 757px de
largura.

A propriedade margin define as margens desse elemento.
Entende-se margem como sendo o espaçamento entre o topo,
lado direito, lado esquerdo e abaixo de um elemento.
Quando a margem está automática significa que o conteúdo
fica centralizado na página.
*/
header {
    width: 757px;
    margin: auto;
}

/*

```

A propriedade background-color define a cor de fundo do elemento nav em cinza. Assim, as cores em folha de estilo (CSS) são definidas em Hexadecimais.

A propriedade height define a altura em 25px, dessa forma define uma espessura a barra criada em folha de estilo.

A propriedade margin-top define o valor de 20px, ou seja está à 20px em relação ao topo do navegador neste caso.

```

*/
nav {
    background-color: #6d7276;
    height: 25px;
    margin-top: 20px;
}

/*
A propriedade float define o alinhamento do elemento que
está com o ID #logotipo. Assim, quando o valor da
propriedade é igual a left indica que o elemento é
lançado à esquerda.

*/
#logotipo {
    float: left;
}

/*
Uma lista é forma por lista de itens e cada item deve ser
alinhado a esquerda. Quando a propriedade left é
atribuída ao elemento li significa esse comportamento.
*/
#menu li {
    float:left;
}

/*
Quando encontrar um ID definido como #menu e que tenha a
tag a é aplicado as seguintes regras:
    - Atribuir um padding de 5px margem top;
    - Atribuir um padding de 5px margem bottom;
    - Atribuir um padding de 12px margem right;
    - Atribuir um padding de 12px margem left;
    - Atribuir a cor em branco;
    - Atribuir a aparência em bloco;
*/
#menu a {
    padding: 5px 12px;
    color: #FFFFFF;
    display:block;
}

/*
Quando o usuário passa o mouse em cima do link aciona a
propriedade hover. Assim, a cor da fonte deve ser preta
(#000000), a cor de fundo branca (#FFFFFF) e deve ser
apresentado como bloco (display:block). */

```

```

#menu a:hover {
    background-color:#FFFFFF;
    color:#000000;
    display:block;
}

/*
Configurando a posição dos campos de entrada do Texto
*/

/*
A propriedade width define a largura da tag section em
757px.
A propriedade margin define as margens esquerda, direita,
topo e abaixo com valores automáticos.
A propriedade margin-top define a margem do topo igual a
50px.
*/
section {
    width: 757px;
    margin: auto;
    margin-top: 50px;
}

/*
A propriedade padding define um espaçamento do conteúdo
com as bordas ao retorno do elemento. Neste caso, quando
definido 5 significa que o elemento tem o padding tanto
em cima, abaixo, lado direito e lado esquerdo igual a
cinco.
*/
p {
    padding: 5px;
}

/*
A propriedade text-align define o alinhamento do texto
para direita, esquerda e centro. No contexto que está
sendo tratado o texto está sendo alinhado a direita.

A propriedade width estabelece a largura, neste caso está
sendo definido o tamanho de 75px do elemento label.

A propriedade vertical-align define que o conteúdo deve
ser alinhado no topo do elemento ou alinhado na parte
inferior do elemento. No exemplo, o texto está alinhado
verticalmente ao topo.

A propriedade display simula o comportamento de um
elemento em relação a forma como o elemento deve se
comportar. Ou seja, o elemento label é um elemento in-
line não provocando quebras de linha aos elementos que
seguem o elemento label. No entanto para conseguir
aplicar o tamanho width precisa ser definido a
propriedade inline-block.

```

```

*/
label {
    text-align: right;
    width: 75px;
    vertical-align: top;
    display: inline-block;
}

/*
A propriedade width define a largura de 400px de uma tag
input que tenha uma classe definida como sendo texto.

A propriedade font-size modifica o tamanho da fonte
presente no elemento input. A unidade em está relacionado
ao tamanho da letra M em maiúsculo sendo variável assim
como a definição em porcentagem.
*/
input.texto {
    font-size: 1em;
    width: 400px;
}

/*
A propriedade font-size modifica o tamanho da fonte
presente no elemento input. A unidade em está relacionado
ao tamanho da letra M em maiúsculo sendo variável assim
como a definição em porcentagem.
*/
input.textarea {
    font-size: 1em;
}

/*
A propriedade padding define um espaçamento do conteúdo
com as bordas ao retorno do elemento. Neste caso, quando
definido 5 significa que todos os elementos tem o padding
tanto em cima, abaixo, lado direito e lado esquerdo igual
a cinco.

A propriedade color define a cor do texto pertencente ao
elemento legend em azul.
*/
legend {
    padding: 5px;
    color: #006AD6;
}

```

Ao clicar no botão enviar apresenta mensagens ao usuário solicitando que seja informado algum dado nos campos nome e mensagem, veja um exemplo na Figura 16.

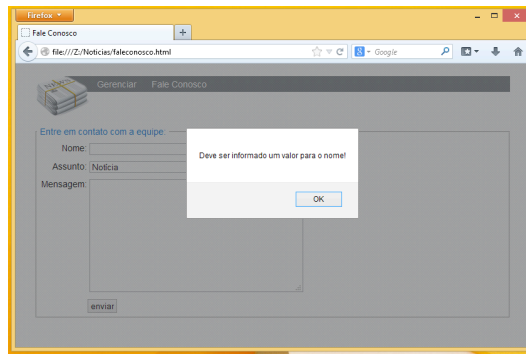


Figura 16 - Mensagem solicitando que dado seja informado
2.5 Considerações finais

Essa unidade introduziu o HTML e o uso de formulários para permitir a interação entre aplicações clientes e servidores Web. Foram abordados assuntos referentes à criação, manipulação e alteração de páginas com HTML, a manipulação da interface gráfica utilizando **Folhas de Estilos (CSS)** e o uso de **JavaScript** para adicionar comportamentos a documentos HTML.

2.6 Estudos complementares

Deitel, H. M.; Deitel, P.J. – Java Como Programar. 6ª. Edição. Editora Pearson- Prentice Hall, 2005.

Gonçalves, E. – Desenvolvendo Aplicações Web com NetBeans IDE 5.5 - Editora Ciência Moderna, 2007.

Fields, D.K.; Kolb, M.A. – Desenvolvendo na Web com JavaServer Pages – Editora Ciência Moderna, 2000.

Serson, R.R. – Certificação Java 5. Brasport Livros e Multimidia Ltda, 2006.